ADR-22 — Dashboard e ESG

Architecture Decision Record · Projeto Steely · Escola de TI · UniCesumar

Status	☒ Proposto ☐ Aceito ☐ Rejeitado ☐ Substituído pelo ADR-___	**Data**	31/10/2026
Responsável	Pietro Pasqual Silva	**Sprint**	8–9 / 15–17
Participantes	Fabrício Notoya Thomas, Augusto Palma Guglielmi, Thiago Lopes Martins, Pietro Pasqual Silva, Rafael Tudela Rizental		
Vínculos	RF-19 · EDTS-52 · STY054 · RNF-04 · RNF-12 · ADR-04 · ADR-08 · ADR-09 · ADR-13 · ADR-16 · ADR-17 · ADR-18 · ADR-20		

1. Contexto — obrigatória

Qual é o problema ou a necessidade? Por que decidir agora? Descreva a situação atual e as restrições (prazo, custo, equipe, stack, LGPD).

O RF-19 pede um painel ESG com resíduo eletrônico evitado (kg), CO2 evitado e minerais preservados, em visão individual (perfil) e consolidada da plataforma. Hoje o dashboard usa dados mockados no front, então os números não refletem as transações reais.

O STY054 exige descartar devoluções, usar a média da categoria quando faltar peso (marcando como estimativa) e versionar a metodologia. O RNF-12 pede fatores versionados com memória de cálculo exportável, e o RNF-04 limita leituras a p95 ≤ 200 ms. A stack é FastAPI sobre PostgreSQL no Supabase (ADR-08), com RLS ativo (ADR-09) e pg_cron como agendador (ADR-16).

2. Direcionadores da decisão — recomendada

Critérios que pesam na escolha: segurança e privacidade, custo (plano gratuito), esforço até 23/11, facilidade de demonstrar na defesa, manutenção.

- Números confiáveis e auditáveis, calculados em um só lugar.
- Desempenho dentro do RNF-04 mesmo com muitas transações.
- Nenhum usuário vê dados de outro (ADR-09).
- Gráficos acessíveis e responsivos (ADR-20) com as cores dos tokens (ADR-19).
- Biblioteca gratuita, leve e fácil de usar em React; painel convincente para a defesa.

São duas escolhas: onde agregar os dados e com qual biblioteca desenhar.

3. Opções consideradas — obrigatória

Liste no mínimo duas opções, cada uma com descrição, prós e contras.

Agregação A — No servidor

Descrição

Views materializadas no PostgreSQL usam a tabela de fatores versionada; o pg_cron atualiza e o FastAPI devolve séries prontas.

Prós

- Cálculo único para web, app e relatórios
- Payload pequeno; atende RNF-04 e RNF-12
- Não expõe dados de outros usuários

Contras

- Exige SQL e job de atualização

Agregação B — No cliente

Descrição

O FastAPI devolve as transações brutas e o front soma e converte.

Prós

- Simples de começar
- Sem trabalho no banco

Contras

- Payload grande e lento
- Lógica duplicada no app
- Vazaria dados de terceiros na visão consolidada

Gráficos A — Recharts

Descrição

Componentes React declarativos que desenham em SVG.

Prós

- Sintaxe JSX natural
- SVG acessível e responsivo
- Fácil aplicar tokens

Contras

- Mais lento com milhares de pontos (não é o caso)
- Não roda em React Native

Gráficos B — Chart.js

Descrição

Biblioteca em canvas com wrapper react-chartjs-2.

Prós

- Leve e rápida
- Boa documentação

Contras

- Canvas é invisível para leitor de tela
- Configuração por objetos, menos "React"

Gráficos C — Apache ECharts

Descrição

Biblioteca completa em canvas/SVG com muitos tipos de gráfico.

Prós

- Muito poderosa
- Mapas e grandes volumes

Contras

- Pacote grande, que pesa no Lighthouse
- Curva maior

ADR-22 — Dashboard e ESG

Continuação

4. Decisão

obrigatória

*Opção escolhida: **Agregação A + Gráficos A**. Justifique por que atende melhor aos direcionadores e por que as outras foram descartadas.*

Agregação no servidor (PostgreSQL + pg_cron, exposta pelo FastAPI) e Recharts no web. O mock é removido.

Agregar no servidor é a única forma de cumprir RNF-12, RNF-04 e a privacidade da visão consolidada. O Recharts entrega gráficos SVG acessíveis e fáceis de estilizar com os tokens. Chart.js foi descartado pela acessibilidade do canvas e ECharts pelo tamanho do pacote.

Endpoint (FastAPI)	Retorno
`GET /api/v1/esg/me?de=&ate=`	Totais do usuário do JWT (kg, CO2e, minerais), série mensal e selos
`GET /api/v1/esg/plataforma?de=&ate=&regiao=`	Totais consolidados, série mensal e quebra por categoria e região
`GET /api/v1/esg/metodologia`	Versão vigente dos fatores e memória de cálculo para exportação

- As views ficam no schema `esg`, sem permissão de leitura para o papel `authenticated`, porque views materializadas não aplicam RLS. O FastAPI abre a sessão com RLS (`SET LOCAL ROLE authenticated` e claims do JWT, ADR-09) e lê os dados por funções `SECURITY DEFINER` no schema `private` (`private.esg_resumo_usuario`, `private.esg_resumo_plataforma`) que filtram por `auth.uid()`, com `search_path = public, private, pg_temp`, `EXECUTE` revogado de `public` e concedido só a `authenticated`; a Data API fica desabilitada (ADR-09, ADR-14).
- O pg_cron roda `REFRESH MATERIALIZED VIEW CONCURRENTLY` a cada 1 hora e no fechamento do período (ADR-16).
- A quebra por região só exibe regiões com ao menos 30 transações no período, no mesmo critério do ADR-04.
- Respostas tipadas em Pydantic (ADR-13) em camelCase com `versaoMetodologia` e `estimado` quando algum item usou a média da categoria.
- O ESG fica no projeto transacional (ADR-08), não no projeto DaaS do ADR-04, porque usa dados por usuário.
- Escopo: web. Gráficos no app ficam fora deste ADR, porque o Recharts não roda em React Native.

5. Consequências

obrigatória

Positivas · Negativas ou riscos aceitos · O que muda no código, no banco, na arquitetura e nos diagramas.

Positivas

- Painel com dados reais e o mesmo número no web, no app e no perfil.
- Metodologia auditável e exportável (RNF-12); respostas pequenas e rápidas.

Negativas ou riscos aceitos

- Visão consolidada atrasa até a próxima atualização (no máximo 1 hora).
- Cold start do Render gratuito ameaçaria o p95; mitigado pelo UptimeRobot chamando `/api/v1/health` a cada 10 min, com GitHub Actions de reserva (ADR-13, ADR-16).
- Fatores de conversão iniciais são estimativas e precisam de fonte citada.
- A demo precisa de transações de seed para o painel não aparecer vazio.

O que muda

- Banco: tabela `esg_fatores` (categoria, massa_kg, co2e_kg, minerais, versao, vigente_desde); views `esg.usuario_mensal` e `esg.plataforma_mensal` sem devoluções concluídas; funções de leitura no schema `private`; job de refresh no pg_cron (a incluir na tabela de prazos do ADR-16).
- Backend: router `esg` no FastAPI com schemas Pydantic.
- Front: remover o mock; hooks `useEsgMe` e `useEsgPlataforma` com TanStack Query e tipos gerados (ADR-18); Recharts com tokens e tabela alternativa (ADR-20).
- Diagramas: ER com `esg_fatores` e schema `esg`; componentes com o fluxo PostgreSQL → FastAPI → dashboard.

6. Validação

recomendada

Como saber se a decisão deu certo? Teste, critério de aceite, métrica ou demonstração que comprova.

- pytest + HTTPX (ADR-17): com transações conhecidas, kg e CO2e batem com o cálculo manual; devolução concluída some do total.
- Usuário A não consegue obter os dados ESG do usuário B; região com menos de 30 transações não aparece.
- Teste de ponta a ponta da sessão RLS através do pooler do Supabase: `SET LOCAL ROLE` dentro de transação com asyncpg devolve só os dados do usuário do JWT.

- `GET /api/v1/esg/plataforma` responde em p95 ≤ 200 ms com a base de seed; Lighthouse ≥ 90 no dashboard.
- Nenhuma importação de mock restante no código do dashboard.

7. Plano de implementação

recomendada

Tarefas necessárias, responsáveis e sprint prevista.

Tarefa	Responsável	Sprint
Criar esg_fatores com valores iniciais e fontes	A definir	A definir
Criar schema esg, views materializadas, funções em private e job no pg_cron	A definir	A definir
Implementar o router /api/v1/esg no FastAPI	A definir	A definir
Teste ponta a ponta da sessão RLS via pooler e revisão das funções SECURITY DEFINER	A definir	A definir
Trocar o mock por hooks e gráficos Recharts (EDTS-52)	A definir	A definir
Seed de transações para a demonstração	A definir	A definir

8. Referências

recomendada

Documentação, artigos, diagramas e outros ADRs.

- Recharts — recharts.org
- PostgreSQL — Materialized Views (postgresql.org/docs)
- Supabase — pg_cron (supabase.com/docs)
- RF-19, RNF-04, RNF-12, STY054
- ADR-04, ADR-08, ADR-09, ADR-13, ADR-16, ADR-17, ADR-18, ADR-20

Histórico de revisões

Versão	Data	Autor	Alteração
1.0	31/10/2026	Pietro Pasqual Silva	Criação do ADR
1.1	31/10/2026	Pietro Pasqual Silva	Alinhamento com as decisões do grupo (FastAPI, Supabase, pg_cron)
1.2	31/10/2026	Pietro Pasqual Silva	Fechamento dos conflitos do alinhamento (Vite, snapshot OpenAPI, RFC 9457, RLS no FastAPI)
1.3	31/10/2026	Pietro Pasqual Silva	Segundo alinhamento: limites do Realtime, slowapi, SECURITY DEFINER e sessão RLS
1.5	31/10/2026	Pietro Pasqual Silva	Alinhamento com os ADR-13 a 17 (codes, camelCase, schema private, Idempotency-Key, tela do leilão)
1.6	01/11/2026	Revisão cruzada do grupo	Padronização do nome 'Fabrício' nos participantes, conforme a revisão cruzada.

ADENDO — ADR-22 · Dashboard e ESG

Projeto Steely · Alinhamento final (propostas do Augusto aceitas). Itens do grupo aplicados a este documento.

Cronograma (item 18)

- Janela de entrega (item 18): Sprints 8–9 / 15–17, até 23/11/2026.
- O campo Sprint do cabeçalho passa a registrar essa janela.

Limiar k e parâmetros compartilhados (itens 13 e 16)

- O limiar k ≥ 30 transações do painel ESG usa o mesmo parâmetro em app_config do ADR-04. As bases ficam separadas: Core (schema esg.*) e Projeto 2 DaaS (ADR-04).
- A quebra por região também exige COUNT(DISTINCT vendedor_id) ≥ 3 no período, parâmetro em app_config, no mesmo critério do ADR-04.

Dados de demonstração (item 19)

- A seed de transações do painel (seção 7) é rotulada "DEMONSTRAÇÃO" e é desligada após a banca, como definido para a massa sintética.

Histórico: versão 1.7 — aplicação dos itens 12, 13, 18 e 19 do alinhamento final do grupo. Este adendo registra as decisões de negócio do grupo que afetam este ADR; o texto das seções 1 a 8 permanece como na versão anterior.